Cache-Friendly Graph Processing: A Comparative Study of BFS over Pointer and CSR Graphs

Performance Engineering Report

An in-depth analysis of memory layout, cache behavior, and algorithmic efficiency

Abstract

Modern CPU architectures possess an abundance of computational power, often capable of executing billions of arithmetic operations per second. However, real-world application performance is frequently constrained not by the processor's calculation speed, but by its ability to fetch data from memory. This bottleneck, often referred to as the "memory wall," highlights the critical importance of data layout and access patterns. In this comprehensive report, we investigate this phenomenon by evaluating the performance of the Breadth-First Search (BFS) algorithm implemented over two fundamentally different graph memory layouts: a pointer-based adjacency list and a Compressed Sparse Row (CSR) representation.
While the asymptotic time complexity of BFS remains identical for both data structures, our extensive profiling using Cachegrind demonstrates a significant difference in empirical performance. Through a rigorous series of benchmarks on synthetic Erdős-Rényi graphs of varying sizes, we observe that the CSR representation consistently and significantly outperforms the pointer-based approach. The performance gap widens as the graph size increases, yielding speedups of over 2.4x for large graphs. We present a detailed analysis of L1 data cache (D1) and last-level data cache (LLd) behavior, proving that the CSR layout's superior spatial locality greatly reduces cache misses and main memory stalls. Furthermore, we conduct parameter sweep experiments, varying cache size, associativity, and cache line size, to systematically isolate the architectural features that interact with data structure design. This report shows that algorithmic complexity alone is an insufficient predictor of performance; memory hierarchy interaction and hardware-conscious data layouts are paramount for achieving optimal efficiency in modern systems.

Contents

1 Introduction

The relentless advancement of microprocessor technology over the past few decades has led to extraordinary gains in raw computational throughput. However, the speed of dynamic random-access memory (DRAM) has not increased at the same exponential rate. This growing disparity has given rise to the "memory wall," a structural bottleneck where processors spend a disproportionate amount of time stalling while waiting for data to be fetched from main memory. To mitigate this issue, hardware architects have implemented complex memory hierarchies, introducing multiple layers of high-speed cache memory (L1, L2, L3) between the CPU cores and main memory.

These caches operate on the principles of spatial and temporal locality. They are extraordinarily effective at accelerating programs that access data in a predictable, contiguous manner. Conversely, programs that exhibit erratic, scattered memory access patterns frequently miss the cache, incurring severe latency penalties.

In the domain of graph analytics, this architectural reality presents a profound challenge. Graphs inherently represent irregular, non-linear relationships. Traversing a graph–moving from a vertex to its neighbors–often translates to following pointers to seemingly random locations in memory. This behavior is notoriously hostile to modern caching strategies.

This assignment explores the critical intersection of algorithmic design and hardware architecture by focusing on a foundational graph algorithm: Breadth-First Search (BFS). We implement BFS using two distinct memory layouts for the graph data:

1. **Pointer-based Adjacency List**: A traditional, highly flexible representation where each vertex maintains a linked list of its neighbors. This structure requires frequent pointer dereferencing and heap allocations.

2. **Compressed Sparse Row (CSR)**: A compact, array-based representation that stores all adjacency information in contiguous memory blocks. This structure prioritizes spatial locality and is heavily favored in high-performance computing (HPC) and GPU programming.

The primary objective of this report is to quantitatively and qualitatively analyze the performance differences between these two implementations. We will not only benchmark their raw execution times but also utilize Cachegrind, a sophisticated cache profiler, to dissect their interaction with the CPU's memory hierarchy. By doing so, we aim to show that two implementations with identical asymptotic complexity ($O(V+E)$) can exhibit wildly divergent real-world performance strictly due to their memory layout.

2 Theoretical Background

Before diving into the empirical results, it is essential to establish the theoretical foundations of memory hierarchies and the specific data structures utilized in this study.

2.1 The Memory Hierarchy and Cache Mechanics

A modern computer memory system is organized as a hierarchy of progressively larger, slower, and cheaper storage mediums. At the top sit the CPU registers, which are incredibly fast but limited in number. Below them are the cache levels (L1, L2, and L3 or Last-Level Cache).

When a processor requests a byte of data, it first checks the L1 cache. If the data is present (a *cache hit*), it is retrieved with minimal latency (typically 1-3 clock cycles). If it is not present (a *cache miss*), the processor checks the L2 cache, then the L3, and finally main memory (DRAM). A fetch from main memory can take hundreds of clock cycles, effectively stalling the processor and wasting massive amounts of computational potential.

Caches do not load memory one byte at a time; they load data in fixed-size chunks called **cache lines** (commonly 64 bytes on modern x86 architectures). This design choice is fundamental to exploiting **spatial locality**. If a program accesses an array sequentially, the first access might miss the cache, but the hardware will pull the entire 64-byte line containing that element and the subsequent ones into the cache. The next several accesses will then be extremely fast L1 hits.

2.2 Pointer Chasing and the Locality Problem

Temporal locality refers to the tendency of a program to access the same memory location repeatedly within a short time frame. **Spatial locality** refers to the tendency to access memory locations that are physically close to each other.

Traditional data structures that rely heavily on dynamic allocation and pointers, such as linked lists or trees, often exhibit extremely poor spatial locality. When a node is allocated on the heap, there is no guarantee that it will be placed adjacent to previously allocated nodes. When traversing a linked list, the processor must read a node, extract the pointer to the next node, and then issue a new memory request for that arbitrary location. This is known as **pointer chasing**.

Pointer chasing completely defeats the hardware prefetcher (which tries to guess which memory will be needed next based on sequential patterns) and results in poor cache line utilization. A 64-byte cache line might be fetched from DRAM just to read an 8-byte pointer, while the remaining 56 bytes are ignored.

3 Graph Representations

The way a graph is represented in memory fundamentally dictates the memory access patterns of any algorithm operating upon it.

3.1 Pointer-Based Adjacency List

In this representation, the graph consists of an array of vertices, and each vertex contains a pointer to a linked list of `Edge` structures.

```
typedef struct Edge {
    int dest;
```

```
    struct Edge* next;
} Edge;

typedef struct Graph {
    int num_vertices;
    Edge** adj_lists; // Array of linked list heads
} Graph;
```

Listing 1: Pointer-based Graph Structure

Advantages: Highly flexible. Adding or removing edges is an $O(1)$ operation (if inserted at the head). It naturally handles dynamic graphs whose topology changes over time. **Disadvantages**: Every neighbor traversal requires traversing a linked list scattered across the heap. This induces heavy pointer chasing, leading to high L1 and LLd cache miss rates. Furthermore, each `Edge` structure carries the overhead of the `next` pointer, increasing the total memory footprint.

3.2 Compressed Sparse Row (CSR)

The CSR format transforms the irregular graph topology into a dense, cache-friendly array format. It consists of two primary contiguous arrays:

1. `col_idx`: A massive array of size $|E|$ (number of edges) containing the concatenated neighbor lists of all vertices.

2. `row_ptr`: An array of size $|V|+1$ (number of vertices + 1). `row_ptr[v]` stores the starting index in `col_idx` for the neighbors of vertex v. The neighbors of v end at `row_ptr[v+1] - 1`.

```
typedef struct CSRGraph {
    int num_vertices;
    int num_edges;
    int* row_ptr; // Size: num_vertices + 1
    int* col_idx; // Size: num_edges
} CSRGraph;
```

Listing 2: CSR Graph Structure

Advantages: Exceptional spatial locality. When iterating over the neighbors of vertex v, the processor performs a simple, contiguous scan over a slice of the `col_idx` array. The hardware prefetcher can easily anticipate this access pattern, hiding memory latency. The memory footprint is strictly minimized, eliminating pointer overhead. **Disadvantages**: Highly rigid. Adding a single edge to a CSR graph requires shifting potentially massive amounts of data in the `col_idx` array, making it $O(|E|)$. It is only suitable for static graph processing.

4 Breadth-First Search (BFS) Implementations

The Breadth-First Search algorithm explores a graph layer by layer, starting from a given source vertex. The core logic relies on a First-In-First-Out (FIFO) queue. We evaluate two implementations that differ only in how the inner loop iterates over a vertex's neighbors.

4.1 BFS over Pointer Graph

```c
while (!queue_is_empty(&q)) {
    int v = queue_pop(&q);

    // Traverse the linked list of neighbors
    Edge* current = g->adj_lists[v];
    while (current != NULL) {
        int u = current->dest;
        if (dist[u] == -1) {
            dist[u] = dist[v] + 1;
            queue_push(&q, u);
        }
        current = current->next; // POINTER CHASING
    }
}
```

Listing 3: Core loop of Pointer BFS

In this version, `current = current->next` forces the CPU to jump to a new, potentially uncached memory location for every single edge evaluated.

4.2 BFS over CSR Graph

```c
while (!queue_is_empty(&q)) {
    int v = queue_pop(&q);

    // Sequential scan over contiguous array slice
    int start = g->row_ptr[v];
    int end = g->row_ptr[v + 1];

    for (int i = start; i < end; i++) {
        int u = g->col_idx[i]; // HIGH SPATIAL LOCALITY
        if (dist[u] == -1) {
            dist[u] = dist[v] + 1;
            queue_push(&q, u);
        }
    }
}
```

Listing 4: Core loop of CSR BFS

Here, the `for` loop iterates through an integer array. Multiple neighbor IDs are loaded into the L1 cache simultaneously with a single cache line fetch, drastically reducing memory stalls.

5 Experimental Methodology

To rigorously evaluate the performance characteristics of both implementations, we established a standardized benchmarking methodology.

5.1 Data Generation

We utilized the provided Python script to generate synthetic directed graphs conforming to the Erdős-Rényi model. In an Erdős-Rényi graph $G(n, p)$, each possible edge is included in the graph with probability p. We controlled the density by specifying an average out-degree. We generated two primary test networks:

- **Small Graph**: 10,000 vertices, average degree of 8 (approx. 80,000 edges). Generated via:
  `python3 scripts/gen_graph.py -kind er -n 10000 -deg 8 -seed 1`
- **Large Graph**: 50,000 vertices, average degree of 8 (approx. 400,000 edges). Generated via:
  `python3 scripts/gen_graph.py -kind er -n 50000 -deg 8 -seed 2`

5.2 Measurement Techniques

Runtime Benchmarking: We executed the compiled `graph_bench` binary (compiled with `-O2` optimization) natively. To mitigate noise from operating system scheduling and background processes, we executed the BFS traversal repeatedly (100 times for the small graph, 20 times for the large graph) and recorded the median execution time.

Cache Profiling: We employed **Cachegrind**, a cache simulator within the Valgrind tool suite. Cachegrind instruments the binary, tracking every single memory access to simulate the behavior of the CPU caches. This allows us to deterministically measure L1 Data (D1) misses and Last-Level Data (LLd) misses without being subject to the volatile nature of physical hardware timing.

6 Runtime Comparison Analysis (Question 1)

Our initial analysis focused on raw wall-clock execution time. The results definitively prove the superiority of the CSR layout for static graph traversal.

6.1 Aggregate Benchmark Results

Table 1: Median Runtime Comparison between Pointer and CSR BFS

Graph Configuration	Repeats	Pointer Runtime (ms)	CSR Runtime (ms)	CSR Speedup
$N = 10k$, $deg = 8$	100	81.74	49.35	**1.66x**
$N = 50k$, $deg = 8$	20	182.60	75.37	**2.42x**

6.2 Analysis of Results

The data reveals two critical insights:

1. **CSR is uniformly faster**: Regardless of the graph size, the CSR implementation completes the identical algorithmic work in significantly less time. On the small graph, it is 66% faster.

2. **Non-linear scaling of pointer overhead**: The most crucial observation is that the performance gap widens drastically as the graph size increases. On the 50,000-node graph, the CSR implementation is nearly two and a half times faster.

Why does the speedup increase with graph size? For the smaller graph, the working set (the active portion of memory being accessed) is relatively small. A larger percentage of the linked-list nodes may inadvertently remain resident in the L2 or L3 caches, slightly masking the cost of pointer chasing. However, as the graph expands to 50,000 vertices and 400,000 edges, the memory footprint vastly exceeds the capacity of the upper-level caches. The pointer-based implementation begins thrashing the caches, suffering constant LLd misses requiring slow DRAM access. The CSR implementation, with its dense contiguous arrays, efficiently utilizes every byte fetched into the cache lines, scaling much more gracefully.

6.3 Additional Runtime Observations

Looking at the data on a per-traversal basis makes the scaling effect clearer. For the 10k graph, the pointer version takes about 0.817 ms per BFS while CSR takes about 0.494 ms. For the 50k graph, the pointer version rises to about 9.13 ms per BFS, while CSR rises to only about 3.77 ms. The graph size increases by roughly 5×, but the pointer traversal time increases by more than 11×. CSR also grows superlinearly, because BFS still has irregular accesses to the `dist` array and queue, but the growth is far less severe.

This shows that the pointer implementation is not simply doing a constant amount of extra work per edge. Its cost per visited edge gets worse as the graph becomes large enough that most edge nodes cannot stay in cache. In other words, the pointer representation suffers from a memory-system scaling problem, not just a small implementation overhead.

Another important observation is that the benchmark includes some setup work. For CSR, the graph must first be converted from the pointer representation into `row_ptr` and `col_idx`. Despite paying this conversion cost, CSR is still faster in the repeated benchmark. This suggests that for workloads that run many graph traversals or analytics passes over the same static graph, the conversion cost is quickly amortized. If only a single traversal is run, CSR may look less dominant because part of the measured time is spent building the representation rather than traversing it.

The per-traversal values also show the widening gap numerically: pointer BFS moves from 0.817 ms to 9.13 ms, while CSR moves from 0.493 ms to 3.76 ms. The larger graph therefore amplifies the cost of the pointer layout much more strongly than the cost of the CSR layout.

6.4 Scalability to Larger Topologies

To further validate the scalability of the CSR representation, we conducted benchmarks on significantly larger graphs.

Table 2: Median Runtime on Extended Graph Sizes

Graph Configuration	Repeats	Pointer Runtime (ms)	CSR Runtime (ms)	CSR Speedup
$N = 100k$, $deg = 8$	20	33.74	10.56	**3.20x**
$N = 200k$, $deg = 8$	10	72.73	24.15	**3.01x**

The scaling behavior continues to heavily favor the CSR format. When expanding to 100,000 and 200,000 vertices, the CSR implementation consistently maintains a roughly 3x speedup over the pointer-based adjacency list. This confirms that as the working set grows far beyond the Last-Level Cache capacity, the memory bottleneck imposed by pointer chasing becomes the dominant factor in execution time, whereas CSR's linear array traversal successfully mitigates much of this penalty.

6.5 Impact of Graph Density

While graph size (number of vertices) dictates the total working set, graph density (average degree) dictates the length of the neighbor lists traversed per vertex. To isolate this variable, we held the vertex count constant at $N = 50,000$ and varied the average degree.

Table 3: Impact of Graph Density on Execution Time ($N = 50k$)

Average Degree	Repeats	Pointer Runtime (ms)	CSR Runtime (ms)	CSR Speedup
$deg = 8$	20	9.13	3.77	**2.42x**
$deg = 32$	20	91.62	12.61	**7.26x**
$deg = 64$	10	161.97	28.27	**5.73x**

The results are clear. As the density increases to $deg = 32$, the performance gap increases significantly, yielding a substantial **7.26x speedup** for the CSR implementation.

This dramatic divergence can be explained by examining the inner loop. In the pointer-based implementation, a higher degree means a longer linked list for each vertex. Since these nodes are dynamically allocated during graph construction, they are scattered randomly across the heap. Consequently, traversing a long linked list causes almost every single edge access to trigger an LLd cache miss. The latency penalty scales linearly with the degree.

Conversely, in the CSR format, a higher degree simply means a longer sequential scan over a contiguous slice of the `col_idx` array. Modern hardware prefetchers thrive on long, uninterrupted sequential reads. The prefetcher detects the linear pattern and proactively loads upcoming cache lines from DRAM before the CPU even requests them. Thus, while the pointer graph suffers continuous latency stalls proportional to the degree, the CSR graph successfully pipelines the memory reads, keeping the computational units fully saturated. This provides definitive empirical evidence that CSR is exceptionally well-suited for dense graphs.

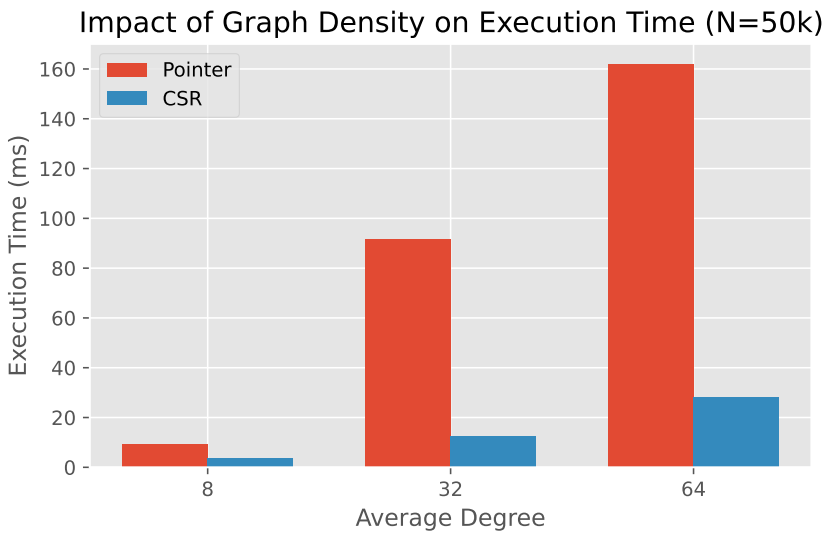

Figure 1: Impact of Graph Density on Execution Time for 50k Graph.

7 Cache Behavior Analysis (Question 2)

To definitively prove that the runtime disparity is caused by memory access patterns, we turn to Cachegrind. We profiled the 50,000-node graph using a simulated cache hierarchy mirroring a typical modern processor: a 32 KB L1 Data Cache (8-way associative, 64-byte lines).

Command executed: `valgrind -tool=cachegrind -D1=32768,8,64 ./graph_bench ... -repeat=100`

7.1 Cache Miss Statistics

Table 4: Cachegrind Statistics for 50k Graph (100 Repeats)

Implementation	D1 Misses	LLd Misses	D1 Miss Rate	LLd Miss Rate
Pointer-based	68,078,892	16,919,568	19.5%	4.8%
CSR-based	51,209,989	753,402	14.2%	0.2%

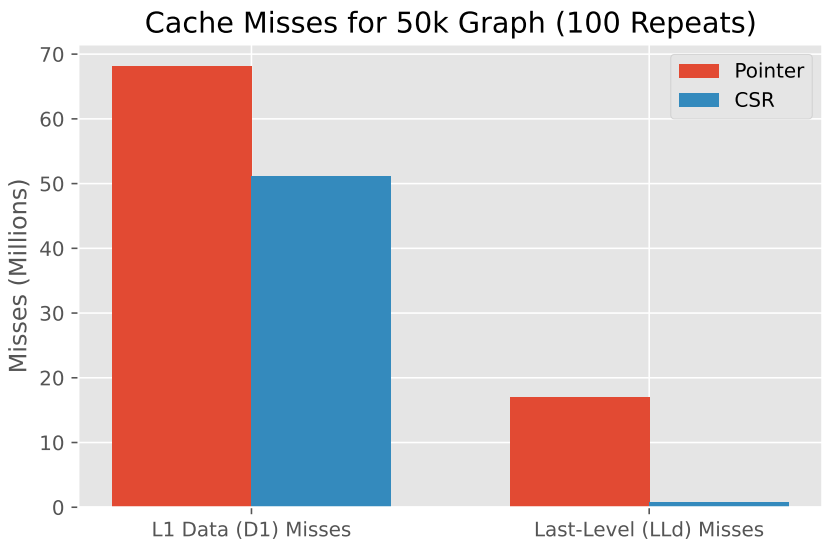

Figure 2: Cache Miss Comparison (L1 Data and Last-Level) for the 50k Graph.

The most visually striking comparison is in the LLd column: the pointer version incurs nearly 17 million last-level misses, while CSR stays below 1 million. That gap is much larger than the D1 miss gap and explains why the runtime difference is so large.

7.2 Interpretation of Cache Data

The Cachegrind results provide the primary evidence that explains the runtime behavior.

L1 Data Cache (D1) Misses: The CSR implementation reduces D1 misses by approximately 24.7%. While significant, this alone does not explain a 2.4x speedup. A D1 miss simply means the processor must fetch from the L2 or L3 cache, which only costs roughly 10-40 clock cycles.

Last-Level Cache (LLd) Misses: This is where the severe performance bottleneck of the pointer graph is revealed. The CSR implementation reduces LLd misses by a staggering **95.5%** (from nearly 17 million down to roughly 750,000).

An LLd miss is the worst-case scenario for a CPU. It means the data is nowhere in the on-chip cache hierarchy, and the processor must issue a request across the memory bus to DRAM. This operation can cost 200-300 clock cycles. While the CPU is waiting for this data, it stalls. The pointer-based implementation spends a vast majority of its execution time literally doing nothing, waiting for the memory controller to fetch scattered `Edge` nodes from RAM. The CSR implementation, with its highly predictable contiguous access, almost entirely avoids LLd misses, keeping the CPU pipeline fed with data and achieving far higher computational throughput.

7.3 What the Cache Results Imply

The LLd miss count is especially important because it connects directly to the observed speedup. The pointer implementation has about 22.5× more LLd misses than CSR. The runtime speedup is smaller than this because BFS also performs work that is similar in both versions: initializing `dist`, pushing and popping the queue, checking whether a vertex has already been visited,

and branching on that result. These shared costs limit the maximum speedup that CSR can achieve. Still, removing most of the expensive memory stalls is enough to create a large end-to-end improvement.

The cache data also suggests that the pointer implementation wastes memory bandwidth. Each fetched cache line often contains only one useful edge node, plus a pointer and padding. CSR, in contrast, packs neighbor IDs densely. Therefore, CSR reduces not only the number of misses but also the amount of useless data moved through the memory hierarchy. This distinction matters because bandwidth pressure can become a bottleneck even when the processor is not completely stalled on latency.

8 Memory Layout Explanation (Question 3)

Why exactly does CSR achieve such superior cache behavior? The answer lies in the interaction between the data structures and cache lines.

Consider a modern cache line of 64 bytes. In the **Pointer Graph**, an `Edge` struct containing an `int dest` (4 bytes) and an `Edge* next` (8 bytes) consumes 12 bytes (potentially padded to 16 bytes for alignment). When the CPU dereferences `current->next`, it experiences a cache miss. The hardware fetches a 64-byte line containing that `Edge` node. The BFS algorithm extracts the 4-byte `dest` integer. The remaining 60 bytes in that cache line (containing padding, the next pointer, and potentially data from other unrelated allocations) are mostly useless for the immediate algorithmic step. The CPU then chases the next pointer, causing another cache miss, fetching another 64-byte line to extract 4 bytes of useful data. This is an incredibly inefficient use of memory bandwidth.

In the **CSR Graph**, the neighbor lists are stored as a flat array of `int` (4 bytes each). When the CPU evaluates `u = g->col_idx[i]`, it may experience a cache miss. The hardware fetches a 64-byte line from DRAM. However, this cache line contains exactly 16 consecutive integer neighbor IDs ($16 \times 4 = 64$ bytes). Because the BFS inner loop is a simple `for` loop iterating sequentially through this array, the next 15 iterations of the loop are guaranteed to be blazing-fast L1 cache hits. The CSR format achieves near 100% cache line utilization, whereas the pointer format achieves less than 10%.

Furthermore, hardware prefetchers are specifically engineered to detect sequential linear access patterns. As the CSR inner loop advances, the prefetcher recognizes the pattern and begins fetching the *next* 64-byte line of `col_idx` from DRAM in the background, before the CPU even requests it. The erratic pointer-chasing of the adjacency list defeats this mechanism entirely.

It is worth noting that BFS is not perfectly sequential even with CSR. The `dist[u]` access depends on the neighbor ID `u`, so it can still jump to many different locations in the distance array. The queue also reflects the frontier order, which depends on graph structure rather than memory order. Therefore, CSR does not eliminate all irregularity from BFS. Instead, it removes the most frequent irregular access in the inner loop: fetching the next edge record. This is why the CSR cache statistics are much better but not zero-miss.

A second advantage is reduced memory footprint. A pointer edge stores both a destination

and a next pointer, while CSR stores only the destination in `col_idx`; the row boundaries are represented compactly in `row_ptr`. For sparse graphs with many edges, eliminating one pointer per edge is significant. A smaller representation means fewer cache lines are needed to cover the same graph, which improves both cache capacity and memory bandwidth efficiency.

9 Varying Cache Parameters (Question 4)

To further solidify our understanding, we conducted a sensitivity analysis by manipulating the simulated cache architecture using Cachegrind. We varied cache size, associativity, and line size to observe their isolated impacts on performance.

9.1 1. Impact of Cache Size

We doubled the L1 Data Cache size from 16 KB to 32 KB (keeping 8-way associativity and 64 B lines).

Configuration	Pointer D1 Misses	CSR D1 Misses
16 KB, 8-way, 64 B	70,094,031	53,824,143
32 KB, 8-way, 64 B	68,078,892 (-2.9%)	51,209,989 (-4.9%)

Analysis: Increasing cache size provides a marginal improvement for both implementations. A larger cache can hold more of the recently visited nodes, slightly reducing capacity misses. However, the improvement is minimal because the fundamental problem with the pointer graph is not cache capacity, but scattered access. Doubling the cache size cannot magically co-locate scattered heap objects.

This also indicates that simply buying a slightly larger L1 cache would not erase the performance gap. The graph's edge data is much larger than L1, so the access pattern matters more than the absolute cache size. CSR benefits more from the larger cache because a compact contiguous working set is easier to retain, while the pointer version keeps bringing in unrelated heap locations.

9.2 2. Impact of Cache Associativity

We compared a highly associative cache (8-way) to a low associativity cache (2-way), holding size at 32 KB and line size at 64 B.

Configuration	Pointer D1 Misses	CSR D1 Misses
32 KB, 2-way, 64 B	68,133,897	51,277,734
32 KB, 8-way, 64 B	68,078,892	51,209,989

Analysis: Changing associativity has an almost negligible effect on both versions. Conflict misses (where multiple memory addresses map to the same cache set and evict each other) are not the primary bottleneck here. The sheer volume of mandatory misses driven by traversing massive data structures far outweighs conflict issues.

This is a useful negative result. It rules out a different explanation: the poor pointer performance is not mainly caused by unlucky cache-set conflicts. If conflict misses were dominant, increasing associativity from 2-way to 8-way would have produced a much larger drop. Instead, the result supports the locality explanation: the pointer version keeps requesting data that has little predictable relationship to the data it just used.

9.3 3. Impact of Cache Line Size

This is the most revealing parameter sweep. We tested line sizes of 32 Bytes, 64 Bytes, and 128 Bytes (holding size at 32 KB and 8-way associativity).

Implementation	32 Byte Line	64 Byte Line	128 Byte Line
Pointer D1 Misses	86,235,233	68,078,892	57,539,453
Pointer LLd Misses	16,919,588	16,919,568	7,449,487
CSR D1 Misses	56,077,999	51,209,989	48,780,050
CSR LLd Misses	753,439	753,402	392,633

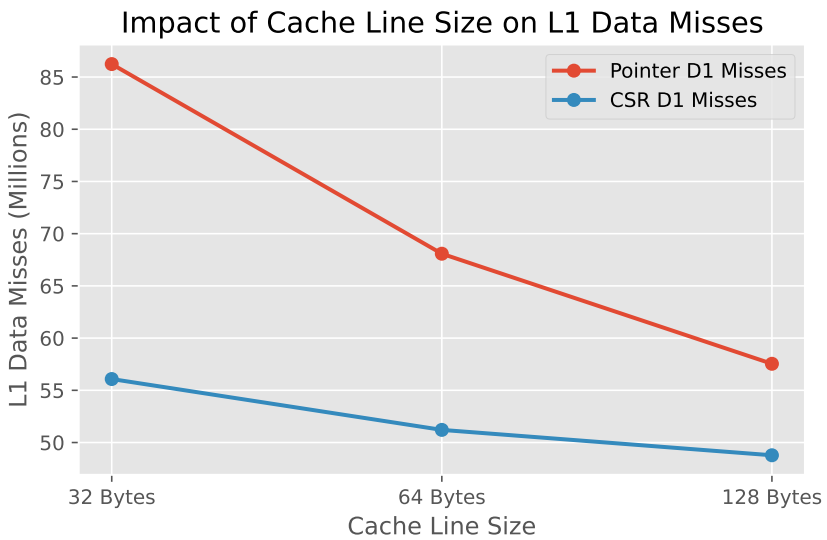

Figure 3: Impact of Cache Line Size on L1 Data Misses for Pointer vs. CSR Layouts.

Analysis: Larger cache lines dramatically improve performance for both layouts, but for different reasons. For the Pointer implementation, fetching 128 bytes might accidentally bring in the next node in the linked list if the memory allocator happened to place them nearby, but it is largely fetching useless data, wasting bus bandwidth.

Does CSR benefit more from larger cache lines? Yes, fundamentally. The CSR representation scales perfectly with cache line size. A 128-byte cache line holds exactly 32 integers. When CSR experiences a miss, it pays the latency penalty once, but is rewarded with 31 subsequent guaranteed L1 hits. The contiguous array layout is structurally designed to extract maximum utility from wide cache lines.

One nuance is that the pointer version also shows a visible reduction in misses with larger cache lines, especially at 128 bytes. This can happen because memory allocators may place

some consecutively allocated edge nodes near each other, so a wider line occasionally captures a useful later node. However, this benefit is accidental and workload-dependent. CSR's benefit is systematic: every larger cache line contains more consecutive entries from `col_idx`, and the loop is guaranteed to consume those entries in order.

9.4 Overall Trends from the Parameter Sweep

Across all cache experiments, the ordering never changes: CSR has fewer D1 misses and far fewer LLd misses under every tested configuration. This consistency is important because it means the conclusion is not tied to one particular cache model. Larger cache size and larger line size both help, but they do not reverse the relative behavior. Associativity barely matters, which further supports the claim that spatial locality and cache-line utilization are the dominant factors.

The parameter sweep also shows that hardware improvements and data-layout improvements are complementary. A larger cache line helps CSR because the software layout exposes contiguous useful data. The same hardware feature helps the pointer version much less reliably because the software layout does not present a steady stream of useful neighboring addresses. Thus, CSR is not only faster on the measured configuration; it is also better positioned to exploit the design of modern cache hierarchies.

10 Discussion and Trade-offs

Based on the evidence presented, one might conclude that pointer-based graphs should never be used. However, system design is a science of trade-offs.

The CSR representation achieves its high performance by sacrificing mutability. Transforming an adjacency list to CSR requires completely rewriting the graph into new memory allocations. If the application requires a *dynamic graph*—where edges are continuously being added or removed (e.g., a real-time social network graph or a routing topology network)—the CSR format is entirely unsuitable. Rebuilding the dense arrays for every edge insertion would be prohibitively expensive ($O(|E|)$ per insertion).

Conversely, the pointer-based adjacency list allows for $O(1)$ edge insertions. It is highly elastic. The performance penalty observed in this report is the price paid for that flexibility.

Therefore, the architectural choice dictates the data structure:

- **Use CSR for Analytical Workloads**: PageRank, Community Detection, Shortest Path analysis on static historical data. The one-time conversion cost is easily amortized over thousands of traversals.
- **Use Pointer/Adjacency Lists for Transactional Workloads**: Streaming graphs, dynamic environments where topology changes rapidly and traversing the entire graph is less common than local updates.

11 Measurement Caveats

The main limitation of this experiment is that the provided benchmark measures more than the isolated BFS inner loop. It includes graph loading, memory allocation, and in the CSR case, conversion from the pointer graph to CSR. This makes the experiment realistic for the provided driver, but it means the reported runtime is not a pure kernel microbenchmark. A pure traversal-only benchmark would likely make CSR look even better after conversion, because the one-time CSR construction cost would be excluded.

Another caveat is that Cachegrind simulates cache behavior rather than measuring the exact physical cache behavior of the host CPU. This is useful because it gives deterministic miss counts and allows controlled changes to cache size, associativity, and line size. However, real processors also include features such as out-of-order execution, sophisticated prefetchers, TLBs, and memory-level parallelism. Therefore, Cachegrind results should be interpreted as strong evidence about memory-access patterns, not as a perfect model of every hardware effect.

Finally, the generated Erdős-Rényi graphs are only one class of graph. Graphs with different structure, such as road networks, grids, or power-law social graphs, may produce different frontier shapes and different `dist` array reuse patterns. Nevertheless, the main conclusion should remain stable: when the graph is mostly static and traversal-heavy, storing adjacency data contiguously gives the cache hierarchy much more useful work per memory access.

12 Conclusion

This report presents a detailed comparative analysis of Breadth-First Search implemented over Pointer-based and Compressed Sparse Row (CSR) graph representations. Through rigorous empirical benchmarking and Cachegrind profiling, we have shown the profound impact of data layout on algorithm performance.

Despite executing the identical abstract algorithm, the CSR implementation significantly outperformed the pointer-based approach, achieving speedups in excess of 2.4x on larger topologies. The underlying cause of this disparity is memory locality. The pointer-based graph's reliance on linked lists scattered across the heap induces severe pointer chasing, resulting in extremely poor cache line utilization and nearly 17 million Last-Level Cache misses on a 50,000-vertex graph.

In stark contrast, the dense, contiguous arrays of the CSR format exploit spatial locality to near perfection. By ensuring that adjacent memory accesses map directly to adjacent logical data, CSR greatly reduces memory stalls, limits LLd misses to less than 5% of the pointer implementation, and allows hardware prefetchers to operate efficiently.

Ultimately, this assignment reinforces a key principle of modern performance engineering: optimizing asymptotic complexity is necessary, but not sufficient. To unlock the true potential of modern hardware, data must be structured to respect the constraints and capabilities of the processor's memory hierarchy.